

Excluding routes

At times, we may want to **exclude** certain routes from having middleware applied. This can be easily achieved using the `exclude()` method. The `exclude()` method accepts a single string, multiple strings, or a `RouteInfo` object to identify the routes to be excluded.

Here's an example of how to use it:

```
consumer
  .apply(LoggerMiddleware)
  .exclude(
    { path: 'cats', method: RequestMethod.GET },
    { path: 'cats', method: RequestMethod.POST },
    'cats/{*splat}',
  )
  .forRoutes(CatsController);
```

Multiple middleware

As mentioned above, in order to bind multiple middleware that are executed sequentially, simply provide a comma separated list inside the `apply()` method:

```
consumer.apply(cors(), helmet(), logger).forRoutes(CatsController);
```

Global middleware

If we want to bind middleware to every registered route at once, we can use the `use()` method that is supplied by the `INestApplication` instance:

main.ts

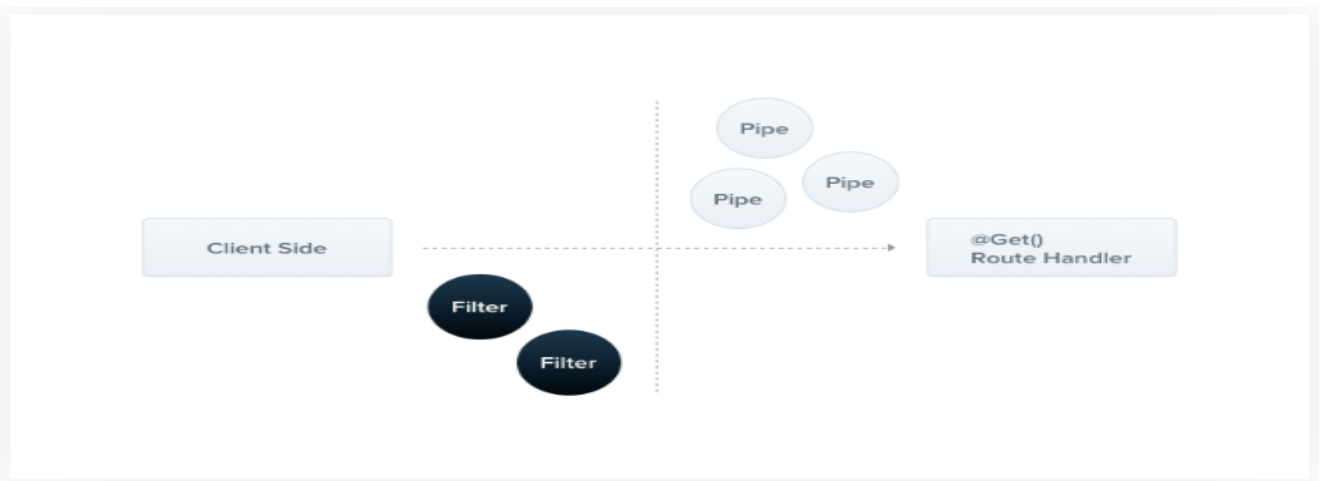
JS

```
const app = await NestFactory.create(AppModule);
app.use(logger);
await app.listen(process.env.PORT ?? 3000);
```

HINT

Accessing the DI container in a global middleware is not possible. You can use a **functional middleware** instead when using `app.use()`. Alternatively, you can use a class middleware and consume it with `.forRoutes('*')` within the `AppModule` (or any other module).

Nest comes with a built-in **exceptions layer** which is responsible for processing all unhandled exceptions across an application. When an exception is not handled by your application code, it is caught by this layer, which then automatically sends an appropriate user-friendly response.



Out of the box, this action is performed by a built-in **global exception filter**, which handles exceptions of type `HttpException` (and subclasses of it). When an exception is **unrecognized** (is neither `HttpException` nor a class that inherits from `HttpException`), the built-in exception filter generates the following default JSON response:

```
{
  "statusCode": 500,
  "message": "Internal server error"
}
```

HINT

The global exception filter partially supports the `http-errors` library. Basically, any thrown exception containing the `statusCode` and `message` properties will be properly populated and sent back as a response (instead of the default `InternalServerErrorException` for unrecognized exceptions).

```
*****

import { Body, Controller, Get, HttpException, HttpStatus, Post } from
 '@nestjs/common';

@Controller('cats')
export class CatsController {
  @Get()
  async findAll(): Promise<Cat[]> {
    // return this.catsService.findAll();
    // throw new HttpException('FORBIDDEN', HttpStatus.FORBIDDEN);
    throw new HttpException(
      { status: HttpStatus.FORBIDDEN, message: 'This is custom message'},
      HttpStatus.FORBIDDEN,
      { cause: 'Custom Reason'}
    )
  }
}

*****
```

The `HttpException` constructor takes two required arguments which determine the response:

- The `response` argument defines the JSON response body. It can be a `string` or an `object` as described below.
- The `status` argument defines the **HTTP status code**.

By default, the JSON response body contains two properties:

- `statusCode` : defaults to the HTTP status code provided in the `status` argument
- `message` : a short description of the HTTP error based on the `status`

To override just the message portion of the JSON response body, supply a string in the `response` argument. To override the entire JSON response body, pass an object in the `response` argument. Nest will serialize the object and return it as the JSON response body.

The second constructor argument - `status` - should be a valid HTTP status code. Best practice is to use the `HttpStatus` enum imported from `@nestjs/common`.

There is a **third** constructor argument (optional) - `options` - that can be used to provide an error **cause**. This `cause` object is not serialized into the response object, but it can be useful for logging purposes, providing valuable information about the inner error that caused the `HttpException` to be thrown.

Exceptions logging

By default, the exception filter does not log built-in exceptions like `HttpException` (and any exceptions that inherit from it). When these exceptions are thrown, they won't appear in the console, as they are treated as part of the normal application flow. The same behavior applies to other built-in exceptions such as `WsException` and `RpcException`.

These exceptions all inherit from the base `IntrinsicException` class, which is exported from the `@nestjs/common` package. This class helps differentiate between exceptions that are part of normal application operation and those that are not.

Built-in HTTP exceptions

Nest provides a set of standard exceptions that inherit from the base `HttpException`. These are exposed from the `@nestjsjs/common` package, and represent many of the most common HTTP exceptions:

- `BadRequestException`
 - `UnauthorizedException`
 - `NotFoundException`
 - `ForbiddenException`
 - `NotAcceptableException`
 - `RequestTimeoutException`
 - `ConflictException`
 - `GoneException`
 - `HttpVersionNotSupportedException`
 - `PayloadTooLargeException`
 - `UnsupportedMediaTypeException`
 - `UnprocessableEntityException`
 - `InternalServerErrorException`
 - `NotImplementedException`
 - `ImATeapotException`
-
- `MethodNotAllowedException`
 - `BadGatewayException`
 - `ServiceUnavailableException`
 - `GatewayTimeoutException`
 - `PreconditionFailedException`

```
@Get()
async findAll(): Promise<Cat[]> {
  throw new BadRequestException('Error-01',
    { cause: new Error(), description: 'Simple description about Error-01' }
  )
}
```

To use exception filters:

```
import { ArgumentsHost, Catch, ExceptionFilter, HttpException } from
 '@nestjs/common';
import { Request, Response } from 'express';

@Catch(HttpException)
export class CatsFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp()
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();
    const status = exception.getStatus();

    response.status(status).json({
      statusCode: status,
      timestamp: new Date().toISOString(),
      path: request.url,
    })
  }
}
```

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { CatsFilter } from './cats/cats.filter';

async function bootstrap() {
  const app = await NestFactory.create(AppModule)
  app.enableCors()
  app.useGlobalFilters(new CatsFilter())
  await app.listen(process.env.PORT ?? 3000)
}
bootstrap();
```

The `@Catch(HttpException)` decorator binds the required metadata to the exception filter, telling Nest that this particular filter is looking for exceptions of type `HttpException` and nothing else. The `@Catch()` decorator may take a single parameter, or a comma-separated list. This lets you set up the filter for several types of exceptions at once.

```
import { BadRequestException, Controller, Get, UseFilters } from
 '@nestjs/common';
import { CatsFilter } from './cats.filter';

@Controller('cats')
export class CatsController {
  @Get('/catsFilter')
  @UseFilters(new CatsFilter())
  async getException() {
    throw new BadRequestException()
  }
}

*****
```

We have used the `@UseFilters()` decorator here. Similar to the `@Catch()` decorator, it can take a single filter instance, or a comma-separated list of filter instances.

```
*****

@Get('/catsFilter')
// @UseFilters(new CatsFilter())
@UseFilters(CatsFilter)
async getException() {
  throw new BadRequestException()
}

*****
```

HINT

Prefer applying filters by using classes instead of instances when possible. It reduces **memory usage** since Nest can easily reuse instances of the same class across your entire module.

```
*****
```

Exception filters can be scoped at different levels: **method-scoped** of the controller/resolver/gateway, **controller-scoped**, or **global-scoped**.

```
cats.controller.ts JS

@Controller()
@UseFilters(new HttpExceptionHandler())
export class CatsController {}
```

WARNING

The `useGlobalFilters()` method does not set up filters for gateways or hybrid applications.

Catch everything

In order to catch **every** unhandled exception (regardless of the exception type), leave the `@Catch()` decorator's parameter list empty, e.g., `@Catch()`.

```

import { HttpAdapterHost } from '@nestjs/core';

@Catch()
export class CatchEverythingFilter implements ExceptionFilter {
  constructor(private readonly httpAdapterHost: HttpAdapterHost) {}

  catch(exception: unknown, host: ArgumentsHost): void {
    // In certain situations 'httpAdapter' might not be available in the
    // constructor method, thus we should resolve it here.
    const { httpAdapter } = this.httpAdapterHost;

    const ctx = host.switchToHttp();

    const httpStatus =
      exception instanceof HttpException
        ? exception.getStatus()
        : HttpStatus.INTERNAL_SERVER_ERROR;

    const responseBody = {
      statusCode: httpStatus,
      timestamp: new Date().toISOString(),
      path: httpAdapter.getRequestUrl(ctx.getRequest()),
    };

    httpAdapter.reply(ctx.getResponse(), responseBody, httpStatus);
  }
}

```

Acti
Go to

WARNING

When combining an exception filter that catches everything with a filter that is bound to a specific type, the "Catch anything" filter should be declared first to allow the specific filter to correctly handle the bound type.

Inheritance

Typically, you'll create fully customized exception filters crafted to fulfill your application requirements. However, there might be use-cases when you would like to simply extend the built-in default **global exception filter**, and override the behavior based on certain factors.

In order to delegate exception processing to the base filter, you need to extend `BaseExceptionHandler` and call the inherited `catch()` method.

all-exceptions.filter.ts

JS

```
import { Catch, ArgumentsHost } from '@nestjs/common';
import { BaseExceptionHandler } from '@nestjs/core';

@Catch()
export class AllExceptionsFilter extends BaseExceptionHandler {
  catch(exception: unknown, host: ArgumentsHost) {
    super.catch(exception, host);
  }
}
```

WARNING

Method-scoped and Controller-scoped filters that extend the `BaseExceptionHandler` should not be instantiated with `new`. Instead, let the framework instantiate them automatically.

Global filters **can** extend the base filter. This can be done in either of two ways.

The first method is to inject the `HttpAdapter` reference when instantiating the custom global filter:

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  const { httpAdapter } = app.get(HttpAdapterHost);
  app.useGlobalFilters(new AllExceptionsFilter(httpAdapter));

  await app.listen(process.env.PORT ?? 3000);
}

bootstrap();
```

The second method is to use the `APP_FILTER` token **as shown here**.